PDF-Übersetzung ins Deutsche

https://gemini.google.com/app/7456d34d87674535

User prompt: Translate this PDF to german please:

Response: CITYPILOT IA

Integrativer Workshop: „Was machen wir heute?“

- **Dauer:** 150 Minuten
- **Teams:** Mindestens 2 (empfohlen: 2)
- **Abgabe:** GitHub-Repository + 2 Pull Requests pro Studierender/Studierendem sowie eine individuelle KI-Dokumentation (Memoria IA) pro Person.
- **Modus:** GELBE AMPEL (KI als Hilfsmittel erlaubt, transparente und vertretbare Nutzung).

1. Die Geschichte

Ein kleines Tourismusunternehmen möchte einen Prototyp auf den Markt bringen: Ein Besucher gibt eine Stadt und seine Wünsche ein – zum Beispiel eine viertolige Kulturroute durch Valencia mit Kindern. Das System ruft das aktuelle Wetter ab, lokalisiert reale Orte und bittet anschließend ein generatives Modell, eine begründete Reiseroute vorzuschlagen. Jede Anfrage wird protokolliert, und das Unternehmen kann über ein Dashboard einsehen, welche Städte und Aktivitäten am meisten gefragt sind. Es ist ein durchgehend funktionsfähiger Prototyp abzuliefern.

Die Herausforderung besteht nicht darin, einen isolierten Chatbot zu bauen, sondern drei eigene Dienste (Web, PostgreSQL, Jupyter), drei öffentliche Datenabfragen und ein Sprachmodell über eine Flask-API sauber miteinander zu verknüpfen.

2. Was der Benutzer am Ende sehen muss

- **Bildschirm 1: Planer.** Formular mit Stadt (Freitext), Tourismusart (Kultur, Gastronomie, Natur oder Familie), Dauer (2, 4 oder 6 Stunden) und Präferenzen im Freitext (optional: „mit Kindern“, „viel Gehen vermeiden“). Der Button generiert eine Seite mit einer Wetterzusammenfassung, mehreren vorgeschlagenen Orten und einer von Cohere erstellten narrativen Route. Die Orte stammen aus OpenStreetMap, nicht aus der Fantasie des Modells. Zeigen Sie einen klaren Hinweis an, wenn Sie Testantworten verwenden; präsentieren Sie keine Öffnungszeiten oder Veranstaltungen als verifiziert.
- **Bildschirm 2: Analytik.** Dashboard mit Gesamtzahl der Anfragen, am häufigsten abgefragten Städten, Tourismusarten, einer Tabelle mit den letzten 8 Anfragen und mindestens einem aus PostgreSQL generierten Diagramm (z. B. Balkendiagramm nach Stadt oder Art). Kann als Matplotlib-PNG-Bild gerendert werden: JavaScript ist nicht erforderlich. Muss sich bei neuen Abfragen aktualisieren.
- **Eigene API.** `POST /api/planificar` empfängt dieselben Präferenzen im JSON-Format wie das HTML-Formular. `GET /api/estadisticas` liefert die Metriken als JSON. Beide Schnittstellen nutzen denselben Python-Orchestrator, ohne Aufrufe oder Regeln zu duplizieren.
- **PostgreSQL.** Speicherung in der bereitgestellten Tabelle: Benutzereingabe, Datum, Stadt, Art, Dauer, Präferenzen, Koordinaten, Meteorologie, verwendete Orte (JSON), gesendeter Prompt, Modellantwort, Status, Testdaten-Indikator, Modellmodus und Anfragedauer; Fehler müssen ebenfalls verständlich protokolliert werden. Keine Speicherung von API-Schlüsseln.
- **Jupyter.** Vier explizite Tests: Wetterdienst, Flask-API, PostgreSQL-Auslesung und Validierungsfehler. Es reicht nicht aus, Jupyter nur zu starten: Im Notebook müssen ausgeführte Tests sichtbar sein.

3. Zu implementierende Architektur

- Browser –`GET, POST /planificar` → Flask-Web (Dienst `web`)
- Browser –`GET /analitica` → Flask-Web (Dienst `web`)
- Notebook (Dienst `jupyter`) →
  - Open-Meteo/Geocoding
  - Open-Meteo/Forecast
  - OSM / Overpass (Points of Interest)
  - Cohere Chat v2 (oder Test-Mock)
  - PostgreSQL (Dienst `db`)
- Flask –`SELECT` → PostgreSQL
- `web:5000` + `db:5432` + APIs

Docker-Compose-Dienste: `web`, `db` und `jupyter`. Ein Grundgerüst mit Dockerfiles, Abhängigkeiten und Start-SQL ist bereits vorhanden; Sie müssen keine halbe Stunde damit verbringen, die Infrastruktur von Null aufzubauen, aber Sie müssen zeigen, dass Sie verstehen, warum `localhost` für die Kommunikation zwischen Containern nicht ausrecht.

4. Was vorgegeben ist vs. Was Sie tun müssen

Teil	Bereits bereitgestellt	Von Ihnen zu implementieren
Docker	`compose.yaml`, Dockerfiles, `.env.example`	Starten, überprüfen, Fehler beheben und dokumentieren
Öffentliche API	URLs, Parameter, Verträge, JSON-Beispiele	HTTP-Aufrufe, Validierung, Bereinigung und Fehlerbehandlung
Modell	Cohere-Konfiguration + Test-Simulator	Eigener Prompt und echter Chat v2-Aufruf mit dem Dozentenschlüssel
Datenbank	Anfragetabelle, Verbindung und Ping	`INSERT` für jede Anfrage, aggregierte Abfragen und letzte Datensätze
Flask	6 Skelett-Routen, Basisansicht, CSS-Styles	Formular, JSON-API, Orchestrierung, Ergebnisse und Analytik
Jupyter	Notebook mit Aufgabenstellungen für vier Prüfungen	Zellen vervollständigen und mit Nachweisen ausführen
GitHub	Workflow-Anweisungen und vorgeschlagene Aufteilung	2 PRs pro Mitglied, Cross-Review und Merges
Gelbe Ampel	Regeln und Vorlage für individuelle Dokumentation	Rechter Nachweis über KI-Nutzung und Quellen, Entscheidungen, Tests und eigener Beitrag

In jeder Datei gibt es `TODO (ALUMNADO)`. Es wird weder die Lösung noch der Prompt oder das vollständige HTML bereitgestellt. Die Datei `docs/TAREAS_Y_CRITERIOS.md` beschreibt im Detail, was JEDE Funktion tun muss, deren Eingaben/Ausgaben, Fehler und wie sie zu überprüfen ist, jedoch ohne den Code zu schreiben. Es wird geschätzt, wenn Sie begründete Entscheidungen treffen und den Code erklären können, selbst wenn Sie KI-Werkzeuge zur Unterstützung genutzt haben. Wesentliche Hilfestellungen werden individuell in der obligatorischen Dokumentation festgehalten.

5. Empfohlener Zeitplan für 150 Minuten

Minuten	Meilenstein	Signal, dass Sie zum nächsten übergehen können
0–15	Team, Repo, Env, Compose und Aufgabenverteilung	`/api/health` antwortet; Jupyter geöffnet; vereinbarte Branches
15–40	PR 1 jeder Person: externe Dienste	Stadt, Koordinaten, Wetter und normalisierte Liste von Orten; 2 PRs offen und geprüft
40–75	PR 2 jeder Person: HTML, Cohere, DB und Dashboard (gut aufteilen)	Funktionaler Ablauf, auch wenn im Mock; 4 PRs in Arbeit oder gemerged
75–110	Integration und Verbesserungen: Validierungen, Ergebnis, JSON-API, Dashboard	Eine Anfrage erscheint in der DB und auf dem Bildschirm; 1 echtes Diagramm
110–135	Jupyter, Tests und PR-Korrekturen	4 Validierungen im Notebook; ≥ 5 Testanfragen
135–150	Nachweise, Abschluss der Dokumentation und GitHub-Abgabe	Funktionales `main`, 2 PRs/Autor gemerged, finaler README und 1 KI-Dokumentation pro Person

Die Zeit ist als Richtwert gedacht, nicht als Ausweichgrund für die Erweiterung des Projektumfangs. Vervollständigen Sie zuerst den Mindestkreislauf; wenn Sie Zeit übrig haben, wählen Sie Bonusaufgaben. Aufgaben können sich überschneiden. Verbringen Sie keine 40 Minuten damit, das CSS zu verschönern, bevor Sie echte Daten haben.

6. Schritt-für-Schritt-Entwicklung

- **Phase A: Externe Daten.** Implementieren Sie die Funktionen `geocodificar`, `obtener_meteorologia` und `obtener_lugares` in `api_clients.py`. Alle drei empfangen und geben normalisierte Wörterbücher/Listen zurück. Filtern Sie für die Stadt auf Spanien, wenn dies zur Reduzierung von Ambiguitäten hilft; arbeiten Sie für Overpass mit einem begrenzten Radius von 2–3 km. Die Datei `docs/APIS_Y_CONTRATOS.md` gibt URLs, Parameter und Antwortstrukturen an. Unterscheiden Sie zwischen dem Ausbleiben von Ergebnissen und einem HTTP-Fehler; setzen Sie ein Timeout.
- **Phase B: Prompt und Cohere.** Entwickeln Sie in `llm.py` Ihre Prompt-Vorlage (wird nicht mitgeliefert) und den dazugehörigen HTTP-Aufruf. Es wird weder der Text eines vollständigen Prompts noch der finale Cohere-Konsumcode bereitgestellt. Senden Sie: Stadt, Präferenzen, Dauer, Wetter und Liste der bestätigten Ziele mit Koordinaten. Fordern Sie eine Route mit 3–5 Stopps an, die begründet und an eventuelle Niederschläge angepasst ist. Der Prompt muss verbieten, Orte zu erfinden oder zu behaupten, ein Lokal sei geöffnet, wenn dies nicht überprüft wurde. Testen Sie dies: Gleichen Sie mindestens eine Antwort mit den gesendeten Daten ab und notieren Sie in der Dokumentation die initiale Version, deren Ergebnisse sowie eine evaluierte Verbesserung (falls es keine Änderung gab, begründen Sie dies). Der Schlüssel wird ausschließlich in `.env` konfiguriert. Sie können den allgemeinen Ablauf mit `LLM_MODE=mock` testen, aber die vollständige Anforderung erfordert einen authentischen Aufruf des im Unterricht bereitgestellten Modells.
- **Phase C: Integration.** `procesar_solicitud` aus `orchestrator.py` ist das Herzstück des Systems. Es muss validieren, die Schritte sequenziell ausführen, die Zeit messen, Fehler behandeln, das Ergebnis speichern und ein einziges konsistentes Wörterbuch zurückgeben. Die beiden Flask-Routen (HTML und JSON) rufen diese Funktion auf. Lassen Sie das HTML APIs nicht direkt aufrufen.
- **Phase D: Frontend und eigene API.** Schreiben Sie das `<form>` in der Datei `index.html` mit Feldnamen, die dem Vertrag entsprechen. Implementieren Sie `POST /api/planificar` zum Abfangen von `request.form` und `POST /api/planificar` zum Abfangen von `request.get_json()`; beide validieren und nutzen denselben Orchestrator. Füllen Sie `resultado.html` mit dem Wetter, den Orten und dem vom Modell zurückgegebenen Text. Zeigen Sie an, ob das Ergebnis real oder simuliert ist; Sie können Links von jedem Ort zur OpenStreetMap-Karte unter Verwendung ihrer Koordinaten hinzufügen, ohne JavaScript einzubinden.

- **Phase E: Persistenz und Analytik.** Die SQL-Tabelle ist bereits erstellt. Implementieren Sie `guardar_solicitud`, `obtener_estadisticas` und `obtener_ultimas_solicitudes` unter Verwendung von SQL-Parametern und SQLAlchemy; einfache SQL-Anweisungen können verwendet werden. Ersetzen Sie in `analitica.html` die Bindestriche durch Metriken aus der DB, fügen Sie eine Tabelle mit 8 Datensätzen ein und vervollständigen Sie `/analitica/grafico.png` mit Matplotlib. Beim Absenden einer neuen Anfrage sollte ein Aktualisieren der Seite die Ergebnisse verändern.
- **Phase F: Notebook und Validierung.** Öffnen Sie `01_validacion.ipynb` innerhalb des Jupyter-Containers und lösen Sie die vier Übungen. HTTP-Aufrufe an Flask aus dem Notebook erfolgen an `http://web:5000`; SQL-Abfragen nutzen `db:5432`. Fügen Sie dem Projekt außerdem mindestens zwei eigene automatisierte Tests (`pytest`) für Fehler, Verträge oder Normalisierung hinzu (2 Smoke-Tests werden mitgeliefert, zählen jedoch nicht als Ihre eigenen).

7. Reale Risiken, die Sie berücksichtigen müssen

- **Overpass** kann überlastet sein. Machen Sie keine Aufrufe bei jedem Laden des Panels. Ein Timeout und eine nützliche Meldung sind vorzuziehen, als die App zu blockieren. Zum Testen der Pipeline können Sie auf `fixtures/puntos_demo.json` zurückgreifen, klar als synthetisch gekennzeichnet; es ist nicht erforderlich, ein komplexes Wiederholungssystem zu implementieren. Die Backup-Daten ersetzen KEINE Demonstration einer externen Anfrage, wenn der Dienst verfügbar ist.
- **Cohere** benötigt einen Schlüssel. Auch bei kostenloser Evaluierung ist ein Konto/eine Anmeldeinformation erforderlich. Der Dozent stellt einen Schlüssel für die Sitzung zur Verfügung. Die Mock-Option verhindert, dass Studierende Konten eröffnen müssen, und ermöglicht die Durchführung der Übung, falls das Kontingent erschöpft ist; sie entspricht jedoch nicht dem echten Cohere-Konsum.
- **„Was machen wir heute?“** ist nicht „Welche Veranstaltungen gibt es heute?“ Wir bauen eine Route, die an das aktuelle Wetter und kartografierte Orte angepasst ist, keinen Veranstaltungskalender oder Öffnungszeitprüfer. Die Benutzeroberfläche muss darauf hinweisen. Es werden auch keine realen Wegzeiten berechnet; jede Besuchsreihenfolge ist ein Vorschlag, keine verifizierte GPS-Navigation.
- **Sicherheit und Attribution.** Passwörter und Schlüssel gehören nicht nach Git, SQL-Abfragen müssen parametrisiert sein, Stack Traces dürfen dem Benutzer nicht angezeigt werden, ungeescapetes HTML ist zu vermeiden, und Open-Meteo sowie OpenStreetMap-Beitragende müssen zitiert werden. Das gesamte Projekt ist demonstrativ und nicht für die Produktion vorbereitet.

8. Gelbe Ampel: KI-Nutzung und OBLIGATORISCHE Dokumentation

Bei dieser Aktivität dürfen Sie KI nutzen, um zu lernen, zu debuggen, Probleme zu erklären, Lösungen zu erkunden und Ihren Code zu überprüfen, aber das Projekt und die Entscheidungen müssen vertretbar sein. Es wird nicht verlangt zu verbergen, dass Sie Hilfe erhalten haben; es wird verlangt, diese zu dokumentieren und die gesamte Aktivität nicht blind zu delegieren. Es gilt auch nicht als ehrlich, von GitHub, Dokumentation oder einem Kommilitonen kopierten Code ohne Anerkennung als den eigenen auszugeben.

Die Dokumentation ist kein dekorativer Anhang: Sie ist ein individueller Leistungsnachweis und wird bewertet. Jede/r Studierende erstellt eine Kopie von `entrega/MEMORIA_IA_PLANTILLA.md` und speichert sie als `entrega/MEMORIA_IA_<NAME>.md`. Darin müssen enthalten sein: der eigene Beitrag, konsultierte Werkzeuge und Modelle, relevante Prompts oder Anweisungen, erhaltene Antworten/Vorschläge (auch fehlgeschlagene Versuche), was übernommen oder verworfen wurde, wie es überprüft wurde sowie Verknüpfungen zu Commits, PRs und Tests. Besonders wichtig ist die Dokumentation der Entwicklung des App-Prompts und der von der KI vorgeschlagenen oder aus anderen Quellen extrahierten Codefragmente. Wenn jemand keine KI verwendet hat, legt er/sie die Dokumentation dennoch vor und gibt diesen Umstand sowie Quellen und Nachweise eigener Arbeit an.

Eine Notiz wie „Ich habe ChatGPT zur Hilfe genutzt“ ist ungenutztes Material. Hilfreich ist: *„Ich hatte den Fehler X; ich fragte Y; das Modell empfahl Z; ich verwarf einen Teil, weil...; ich änderte diese Funktion; ich überprüfte es mit diesem Test; es erscheint in PR #...“* Es ist nicht erforderlich, jede triviale Frage zu erfassen, aber sehr wohl die substanziellen Interaktionen. Fügen Sie niemals Anmeldeinformationen in Prompts, Screenshots oder Abgabedokumente ein.

- **Mögliche zusätzliche Reduktion:** Eine offensichtlich falsche Dokumentation, die wesentliche KI-/Drittanbieterbeiträge verschleiert oder Tests/Beiträge erfindet, kann zu einer Reduktion der Endnote DIESER Aktivität um bis zu 30 % führen, zusätzlich zu den beim Dokumentationskriterium verlorenen Punkten. Dies gilt nicht für die Nutzung von KI oder bei Verdachtsmomenten aufgrund des Stils oder automatisierter Detektoren: Dokumentation, PRs, Commits, Ergebnisse und persönliche Erklärungen werden abgeglichen und vor einer Entscheidung wird eine Klärung angefordert. Eine weggelassene Dokumentation führt zu 0 Punkten in ihrem Kriterium, entspricht jedoch nicht automatisch einer Fälschung.

Die vollständigen Anweisungen, Beispiele und die Vorlage befinden sich in `docs/SEMAFORO_AMARILLO_Y_MEMORIA.md` und `entrega/MEMORIA_IA_PLANTILLA.md`. Notieren Sie sich die Nachweise während der Entwicklung und erfinden Sie sie nicht erst am Ende.

9. Abgabe und Bewertungsraster (10 Punkte)

Geben Sie **einen** Link zum GitHub-Repository ab, mit funktionalem `main`, eigenem README (Start, was funktioniert, was fehlt), zwei Screenshots (Ergebnis und Dashboard), ausgeführtem Notebook, zwei von Mitgliedern gegengeprüften PRs und einer `.md`-KI-Dokumentation pro Mitglied innerhalb von `entrega/`. Die PRs müssen Autoren und Prüfer ausweisen. Die Abgabe muss die Überprüfung von SQL-Abfragen, Tests und Code ohne Zugriff auf Geheimnisse ermöglichen.

Kriterium	Punkte	Was beobachtet wird
Externe APIs	1,4	Geo + Wetter + Overpass; normalisierte Daten, Fehlerbehandlung
Cohere und Prompt	1,1	Echter Aufruf bei vorhandenem Schlüssel, eigener Prompt, verifizierbare Daten und Anpassung
Flask und Orchestrator	1,5	Formular, JSON-API, Validierung, Ergebnis und geteilte Logik
Persistenz und Analytik	1,6	Vollständiges `INSERT`, PostgreSQL-Dashboard, Tabelle und Diagramm
Docker, Jupyter und Tests	1,3	3 Container, Notebook mit Nachweisen, 2 neue Tests
Git, PR und Kollaboration	1,1	2 PRs pro Person, Cross-Review, reale und zurechenbare Änderungen

Kriterium	Punkte	Was beobachtet wird
Individuelle KI-Dokumentation, wahrheitsgemäß und überprüfbar	2,0	Registrierung 0,8; Kriterium und Entscheidungen 0,6; Nachweise 0,4; persönliche Reflexion 0,2
GESAMT	**10,0**	

- **Bonus bis +1** (maximales Endergebnis vor möglichen Abzügen: 10): Einzelne Links zu OSM (+0,25), dokumentierter Regen/Sonne-Vergleich (+0,25), Overpass-Cache (+0,25), zusätzliches Diagramm nach Stadt/Art (+0,25). Boni ersetzen keine obligatorischen Anforderungen.
- **Wie die 30%-Regelung funktioniert:** Nach Erhalt der Note nach Raster und Bonus (maximal 10) kann eine begründete proportionale Reduktion von 0–30 % angewendet werden, wenn relevante Fälschungen nachgewiesen werden. Beispiel: 8,0 und 20 % Reduktion = 6,4. Dies ist keine 30%-ige Kürzung auf das gesamte Fach und keine automatische Strafe für die Nutzung von KI.

10. Beobachtbare Erfolungskriterien

Am Ende kann das Team folgendes vorweisen:

1. Drei gestartete Dienste;
2. Eine über das Web eingefügte Stadt und Tourismusart;
3. Aktuelle Antworten von Geocoding und Meteorologie;
4. Reale OSM-Punkte und KI-Anpassung der Route;
5. Eine vollständige Zeile in Postgres;
6. Ein Dashboard, das sich nach einer weiteren Abfrage ändert;
7. Notebook mit ausgeführten Tests;
8. Vier PRs mit gegenseitiger Prüfung (Cross-Review);
9. Ehrliche und überprüfbare individuelle KI-Dokumentation jedes/r Studierenden.

Es wird nicht belohnt, mehr Zeilen zu schreiben, sondern den Kreislauf zu schließen und zu verstehen.

Unterstützende Dokumentation: `docs/APIS_Y_CONTRATOS.md`, `docs/TAREAS_Y_CRITERIOS.md`, `docs/GIT_Y_PRS.md`, `docs/COMPROBACIONES.md`, `docs/SEMAFORO_AMARILLO_Y_MEMORIA.md` und `entrega/MEMORIA_IA_PLANTILLA.md`.

User prompt: I have to implement a function obtener_lugares(lat, lon, tip). It looks for POIs via Overpass API. How do I build Overpass-QL requests?

Response: An **Overpass QL** query requires setting output format, defining execution timeout, applying spatial and tag filters, and specifying the output mode.

1. Query Structure

```
[out:json][timeout:15];
(
  nwr[<key>=<value>](around:<radius_in_meters>, <lat>, <lon>);
);
out center;
```

- **`[out:json]`**: Requests the response in JSON format.
- **`[timeout:15]`**: Aborts execution on the Overpass server after 15 seconds.
- **`nwr`**: Queries nodes, ways, and relations simultaneously.
- **`(around:<radius>, <lat>, <lon>)`**: Restricts the search bounding circle in meters.
- **`out center;`**: Outputs element attributes and calculates a single centroid `(lat, lon)` for polygons/ways.

2. OSM Tag Mapping by Category

Map the `tipo` input to standard OSM key-value pairs:

`tipo`	Target OSM Tags (`key` ≈ `value`)
`cultural`	`historic`, `tourism=museum`, `amenity=arts_centre`, `amenity=theatre`
`gastronomico`	`amenity=restaurant`, `amenity=cafe`, `amenity=bar`
`naturaleza`	`leisure=park`, `leisure=garden`, `landuse=forest`

tipo	Target OSM Tags (key ≈ value)
familiar	tourism=zoo, tourism=theme_park, leisure=park, leisure=playground

3. Implementation Pattern (`api_clients.py`)

```
from typing import Any, Dict, List
import requests

OVERPASS_ENDPOINT = "https://overpass-api.de/api/interpreter"

TAG_FILTERS: Dict[str, List[str]] = {
    "cultural": [
        '["historic"]',
        '["tourism"="museum"]',
        '["amenity"="arts_centre"]',
    ],
    "gastronomico": [
        '["amenity"="restaurant"]',
        '["amenity"="cafe"]',
    ],
    "naturaleza": [
        '["leisure"="park"]',
        '["leisure"="garden"]',
    ],
    "familiar": [
        '["leisure"="park"]',
        '["tourism"="zoo"]',
        '["leisure"="playground"]',
    ],
}


def build_overpass_query(
    lat: float, lon: float, tipo: str, radius: int = 2500
) -> str:
    filters = TAG_FILTERS.get(tipo, ['["tourism"]'])
    clauses = "\n  ".join(
        f"nwr{tag_filter}(around:{radius}, {lat}, {lon});"
        for tag_filter in filters
    )

    return f"""[out:json][timeout:15];
(
  {clauses}
);
out center 15;"""


def obtener_lugares(
    lat: float, lon: float, tipo: str
) -> List[Dict[str, Any]]:
    query = build_overpass_query(lat, lon, tipo, radius=2500)
    try:
        response = requests.post(
            OVERPASS_ENDPOINT, data={"data": query}, timeout=15
        )
        response.raise_for_status()
        elements = response.json().get("elements", [])
    except requests.RequestException:
        return []

    lugares = []
    for el in elements:
        tags = el.get("tags", {})
        name = tags.get("name")
        if not name:
            continue

        item_lat = el.get("lat") or el.get("center", {}).get("lat")
        item_lon = el.get("lon") or el.get("center", {}).get("lon")

        if item_lat is not None and item_lon is not None:
            lugares.append(
                {
                    "nombre": name,
                    "lat": item_lat,
                    "lon": item_lon,
```

```
                    "tipo": tags.get("amenity")
                    or tags.get("tourism")
                    or tags.get("historic")
                    or tipo,
                }
            )

    return lugares
```

User prompt: Overpass returns nodes with lat/lon and ways/relations witcenter.lat/center.lon. Some elements don't have a name tag. How do I parseh and normalize this in Python so that I get a list of max 12 unique places without duplicates?

Response:

```
from typing import Any, Dict, List


def parse_and_normalize_overpass(
    elements: List[Dict[str, Any]], limit: int = 12
) -> List[Dict[str, Any]]:
    seen_names = set()
    normalized_places: List[Dict[str, Any]] = []

    for el in elements:
        tags = el.get("tags", {})
        name = tags.get("name", "").strip()

        # Discard elements without a name tag
        if not name:
            continue

        # Extract coordinates depending on element type (node vs way/relation)
        lat = el.get("lat") or el.get("center", {}).get("lat")
        lon = el.get("lon") or el.get("center", {}).get("lon")

        if lat is None or lon is None:
            continue

        # Deduplicate based on lowercase normalized name
        name_key = name.casefold()
        if name_key in seen_names:
            continue

        seen_names.add(name_key)

        category = (
            tags.get("amenity")
            or tags.get("tourism")
            or tags.get("leisure")
            or tags.get("historic")
            or "lugar"
        )

        normalized_places.append(
            {
                "nombre": name,
                "lat": float(lat),
                "lon": float(lon),
                "categoria": category,
            }
        )

        if len(normalized_places) >= limit:
            break

    return normalized_places
```

User prompt: The Overpass API server is often slow or returns 504 Gateway Timeout or 429 too Many Requests. How should I handle exceptions in requests.post to load a local demo fixture as a fallback instead of crashing?

Response:

```
import json
import logging
from pathlib import Path
from typing import Any, Dict, List
import requests

FIXTURE_PATH = Path("fixtures/puntos_demo.json")
OVERPASS_ENDPOINT = "https://overpass-api.de/api/interpreter"
```

```
logger = logging.getLogger(__name__)


def cargar_fixture_demo() -> List[Dict[str, Any]]:
    if not FIXTURE_PATH.is_file():
        logger.error("Demo fixture not found at %s", FIXTURE_PATH)
        return []
    try:
        with open(FIXTURE_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)
            return data.get("elements", []) if isinstance(data, dict) else data
    except (json.JSONDecodeError, OSError) as e:
        logger.error("Failed to read fixture: %s", e)
        return []


def ejecutar_consulta_overpass(query: str, timeout: int = 15) -> tuple[List[Dict[str, Any]], bool]:
    """
    Executes Overpass QL query.
    Returns:
        (elements, es_mock): Tuple containing elements list and a boolean flag
        indicating whether fallback mock data was used.
    """
    try:
        response = requests.post(
            OVERPASS_ENDPOINT,
            data={"data": query},
            headers={"User-Agent": "CityPilotWorkshop/1.0"},
            timeout=timeout,
        )

        # 429 Too Many Requests or 504 Gateway Timeout
        if response.status_code in (429, 504):
            logger.warning("Overpass rate limited/timed out (%s). Falling back to fixture.", response.stat
            return cargar_fixture_demo(), True

        response.raise_for_status()
        data = response.json()
        return data.get("elements", []), False

    except (requests.Timeout, requests.ConnectionError, requests.HTTPError) as err:
        logger.warning("Overpass network/HTTP error (%s). Falling back to fixture.", err)
        return cargar_fixture_demo(), True

    except json.JSONDecodeError:
        logger.warning("Overpass returned invalid JSON (HTML error page). Falling back to fixture.")
        return cargar_fixture_demo(), True
```

Integration Notes

- **Explicit flag:** Returning `es_mock=True` fulfills the requirement to mark test/fallback data in `resultado.html` and in the PostgreSQL `indicador de datos de prueba` column.
- **HTML error detection:** Overpass frequently responds with HTTP 200 or 504 containing an HTML error string instead of JSON; catching `json.JSONDecodeError` prevents crashes during endpoint outages.
- **Timeout threshold:** Keep the `requests.post(..., timeout=timeout)` slightly higher than the Overpass `[timeout:15]` header to allow the remote gateway to cleanly close the connection first.

User prompt: How do I write pytest unit tests with monkeypatch for this method to test both normal parsing of nodes or ways and the fallback behaviour without makin real HTTP requests?

`Response:`
```
import json
from pathlib import Path
from typing import Any, Dict, List
import pytest
import requests

from api_clients import obtener_lugares


class MockResponse:
    def __init__(
        self,
        json_data: Any = None,
        status_code: int = 200,
        text: str = "",
```

```
            raise_exc: Exception | None = None,
        ):
            self._json_data = json_data
            self.status_code = status_code
            self.text = text
            self._raise_exc = raise_exc

        def json(self):
            if isinstance(self._json_data, Exception):
                raise self._json_data
            return self._json_data

        def raise_for_status(self):
            if self._raise_exc:
                raise self._raise_exc
            if self.status_code >= 400:
                raise requests.HTTPError(
                    f"HTTP Error {self.status_code}", response=self
                )


    @pytest.fixture
    def mock_overpass_payload() -> Dict[str, Any]:
        return {
            "elements": [
                # Node element
                {
                    "type": "node",
                    "id": 101,
                    "lat": 39.4699,
                    "lon": -0.3763,
                    "tags": {"name": "Lonja de la Seda", "historic": "monument"},
                },
                # Way element with center coordinates
                {
                    "type": "way",
                    "id": 202,
                    "center": {"lat": 39.4750, "lon": -0.3750},
                    "tags": {"name": "Torres de Serranos", "tourism": "attraction"},
                },
                # Element without name (must be ignored)
                {
                    "type": "node",
                    "id": 303,
                    "lat": 39.4700,
                    "lon": -0.3700,
                    "tags": {"tourism": "museum"},
                },
                # Duplicate name (must be deduplicated)
                {
                    "type": "node",
                    "id": 404,
                    "lat": 39.4699,
                    "lon": -0.3763,
                    "tags": {"name": "lonja de la seda", "historic": "monument"},
                },
            ]
        }


    def test_obtener_lugares_parsing_success(
        monkeypatch: pytest.MonkeyPatch, mock_overpass_payload: Dict[str, Any]
    ):
        def mock_post(*args, **kwargs):
            return MockResponse(json_data=mock_overpass_payload, status_code=200)

        monkeypatch.setattr(requests, "post", mock_post)

        lugares, es_mock = obtener_lugares(39.4699, -0.3763, "cultural")

        assert es_mock is False
        assert len(lugares) == 2
        assert lugares[0]["nombre"] == "Lonja de la Seda"
        assert lugares[0]["lat"] == pytest.approx(39.4699)
        assert lugares[0]["lon"] == pytest.approx(-0.3763)
        assert lugares[1]["nombre"] == "Torres de Serranos"
        assert lugares[1]["lat"] == pytest.approx(39.4750)
        assert lugares[1]["lon"] == pytest.approx(-0.3750)
```

```
@pytest.mark.parametrize(
    "error_kwargs",
    [
        {"raise_exc": requests.Timeout("Connection timed out")},
        {"status_code": 504},
        {"status_code": 429},
        {
            "status_code": 200,
            "json_data": json.JSONDecodeError("Expecting value", "", 0),
        },
    ],
)
def test_obtener_lugares_fallbacks_to_fixture(
    monkeypatch: pytest.MonkeyPatch, error_kwargs: Dict[str, Any], tmp_path: Path
):
    # Mock fallback fixture file
    fixture_content = {
        "elements": [
            {
                "type": "node",
                "id": 999,
                "lat": 39.4700,
                "lon": -0.3700,
                "tags": {"name": "Lugar Demo", "tourism": "museum"},
            }
        ]
    }
    demo_file = tmp_path / "puntos_demo.json"
    demo_file.write_text(json.dumps(fixture_content), encoding="utf-8")

    # Patch FIXTURE_PATH in module
    monkeypatch.setattr("api_clients.FIXTURE_PATH", demo_file)

    def mock_post(*args, **kwargs):
        if "raise_exc" in error_kwargs:
            raise error_kwargs["raise_exc"]
        return MockResponse(**error_kwargs)

    monkeypatch.setattr(requests, "post", mock_post)

    lugares, es_mock = obtener_lugares(39.4699, -0.3763, "cultural")

    assert es_mock is True
    assert len(lugares) == 1
    assert lugares[0]["nombre"] == "Lugar Demo"
```

User prompt: I need to implement guardar_solicitud(registro: dict) gpoing to int in sqlAlchemy Postgresql for my Flask project. The table is called solicitudes with columns: ciudad, pais, tipo_turismo, duracion_horas, preferencias, latitud, longitud meteorologia (jsonB), lugares (jsonB), prompt_usado, itinerario, estado, duracion_ms, error_resumido, modo_demo, modo_modelo. how do I write a safe parameterized INSERT returning the new id, ensuring dicts or lists are properly dumped to jsonB?

Response:
```
import json
from typing import Any, Dict
from sqlalchemy import text


def _serializar_json(val: Any) -> str | None:
    if val is None:
        return None
    if isinstance(val, (dict, list)):
        return json.dumps(val, ensure_ascii=False)
    return str(val)


def guardar_solicitud(db_session, registro: Dict[str, Any]) -> int:
    query = text(
        """
        INSERT INTO solicitudes (
            ciudad,
            pais,
            tipo_turismo,
            duracion_horas,
            preferencias,
            latitud,
            longitud,
            meteorologia,
            lugares,
```

```
            prompt_usado,
            itinerario,
            estado,
            duracion_ms,
            error_resumido,
            modo_demo,
            modo_modelo
        ) VALUES (
            :ciudad,
            :pais,
            :tipo_turismo,
            :duracion_horas,
            :preferencias,
            :latitud,
            :longitud,
            CAST(:meteorologia AS jsonb),
            CAST(:lugares AS jsonb),
            :prompt_usado,
            :itinerario,
            :estado,
            :duracion_ms,
            :error_resumido,
            :modo_demo,
            :modo_modelo
        )
        RETURNING id;
    """
    )

    params = {
        "ciudad": registro.get("ciudad"),
        "pais": registro.get("pais"),
        "tipo_turismo": registro.get("tipo_turismo"),
        "duracion_horas": registro.get("duracion_horas"),
        "preferencias": registro.get("preferencias"),
        "latitud": registro.get("latitud"),
        "longitud": registro.get("longitud"),
        "meteorologia": _serializar_json(registro.get("meteorologia")),
        "lugares": _serializar_json(registro.get("lugares")),
        "prompt_usado": registro.get("prompt_usado"),
        "itinerario": registro.get("itinerario"),
        "estado": registro.get("estado", "completado"),
        "duracion_ms": registro.get("duracion_ms"),
        "error_resumido": registro.get("error_resumido"),
        "modo_demo": bool(registro.get("modo_demo", False)),
        "modo_modelo": registro.get("modo_modelo", "cohere"),
    }

    result = db_session.execute(query, params)
    db_session.commit()
    return int(result.scalar_one())
```

Puntos clave de implementación

- **Casting explícito a JSONB:** `CAST(:columna AS jsonb)` evita incompatibilidades de tipos entre el dialecto del driver PostgreSQL y SQLAlchemy al insertar strings serializados con `json.dumps`.
- **Manejo de `None`:** Si el campo es `None`, se envía `NULL` a nivel de parámetro SQL, permitiendo que `CAST(NULL AS jsonb)` mantenga el valor `NULL` en base de datos sin generar errores de sintaxis.
- **Recuperación del ID:** `RETURNING id` ejecutado con `result.scalar_one()` extrae directamente el entero generado por la secuencia primaria de PostgreSQL.
- **Seguridad:** El enlace de variables por diccionario (`params`) previene inyecciones SQL y garantiza no exponer claves API.

User prompt: My peer requested changes on my PR: 1. rebase conflict in main.py keeping PR #2's /planificar, 2. don't return detalle=str(exc) in /api/estadisticas, 3. isolate the DB test with unique fixtures and cleanup, 4. duracion_horas default 3 is invalid (only 2, 4, 6 allowed). Can you explain what needs to be fixed and why each of these is a problem?

Response: 1. Rebase Conflict in `main.py` Keeping PR #2's `/planificar`

- **The Problem:** Your branch diverged from the base branch (`main`) where PR #2 was already merged, creating conflicting edits in `main.py` around the route definition.
- **Why it matters:** Git cannot automatically determine how to reconcile overlapping modifications. Leaving unresolved conflict markers (<<<<<<<, =======, >>>>>>>) breaks the build. Merging your version over PR #2's would cause a regression by discarding your peer's validated routing or orchestration logic.

- **The Fix:** Run `git fetch origin`, rebase onto the target branch (`git rebase origin/main`), accept the implementation from PR #2 inside `main.py` for `/planificar`, ensure both the HTML form and JSON endpoints call the shared orchestrator, test the routes, and complete the rebase (`git add main.py && git rebase --continue`).

2. Do Not Return `detalle=str(exc)` in `/api/estadisticas`

- **The Problem:** Catching a database or system exception and echoing `str(exc)` directly inside the JSON response payload leaks internal system details.
- **Why it matters:** Returning raw exception strings violates baseline API security guidelines by exposing database table names, column structures, dialect details, connection URIs, or internal driver stack traces to clients. It also breaks contract consistency if clients expect a standard error schema (`{"error": "Error interno del servidor"}`).
- **The Fix:** Log the full exception internally with `logger.exception(exc)` on the server side, and return a sanitized, generic error payload with an appropriate HTTP status code (such as 500):

```
except Exception as exc:
    logger.exception("Error al recuperar estadísticas: %s", exc)
    return (
        jsonify({"error": "No se pudieron obtener las estadísticas"}),
        500,
    )
```

3. Isolate the DB Test with Unique Fixtures and Cleanup

- **The Problem:** The database integration test modifies shared database state without isolating or rolling back inserted rows, or relies on hardcoded primary keys/values that conflict when run in suite execution.
- **Why it matters:** Leaking records into the persistent test table makes tests order-dependent, causes flaky builds when running `pytest` repeatedly or concurrently, and invalidates aggregation assertions (such as row counts or dashboard metrics) in subsequent tests.
- **The Fix:** Use a dedicated pytest fixture that wraps database operations in a transaction and rolls it back after each test, or explicitly executes a teardown deletion:

```
@pytest.fixture
def db_test_session(db_session):
    # Alternatively, delete explicit inserted test rows on teardown
    inserted_ids = []
    yield db_session, inserted_ids
    if inserted_ids:
        db_session.execute(
            text("DELETE FROM solicitudes WHERE id = ANY(:ids)"),
            {"ids": inserted_ids},
        )
        db_session.commit()
```

4. `duracion_horas` Default 3 Is Invalid (Only 2, 4, 6 Allowed)

- **The Problem:** The request parser or schema assigns a default value of `3` when `duracion_horas` is omitted or invalid.
- **Why it matters:** The system contract strictly defines duration as discrete steps: **2, 4, or 6 hours**. Defaulting to `3` injects out-of-spec data into the LLM prompt, fails schema validation if constraints are enforced, and pollutes analytics data with values that cannot be selected in the UI form.
- **The Fix:** Update fallback and validation logic to restrict values to `{2, 4, 6}`. If an invalid or missing value is provided, either default to a valid option (typically `4` or `2`) or reject the input with an HTTP 400 Bad Request error:

```
ALLOWED_DURATIONS = {2, 4, 6}

duracion = data.get("duracion_horas")
if duracion not in ALLOWED_DURATIONS:
    # If treating as validation error:
    return (
        jsonify(
            {
                "error": (
                    "duracion_horas inválida. Valores permitidos: 2, 4, 6"
                )
            }
        ),
        400,
    )
    # Or if providing a fallback compliant with the contract:
    duracion = 4
```

User prompt: How do I implement strict validation for duracion_horas in (2, 4, 6) in db.py, log the exception safely in Flask without exposing str(exc), and write an isolated pytest test that creates a unique row and cleans it up with a DELETE in a finally block

Response: 1. Strict Validation in `db.py`

Enforce allowed values prior to executing the query:

```
# db.py
from typing import Any, Dict
from sqlalchemy import text

ALLOWED_DURATIONS = {2, 4, 6}


def guardar_solicitud(db_session, registro: Dict[str, Any]) -> int:
    duracion = registro.get("duracion_horas")
    if duracion not in ALLOWED_DURATIONS:
        raise ValueError(
            f"duracion_horas must be one of {sorted(ALLOWED_DURATIONS)}, got {duracion!r}"
        )

    query = text(
        """
        INSERT INTO solicitudes (
            ciudad, pais, tipo_turismo, duracion_horas, preferencias,
            latitud, longitud, meteorologia, lugares, prompt_usado,
            itinerario, estado, duracion_ms, error_resumido,
            modo_demo, modo_modelo
        ) VALUES (
            :ciudad, :pais, :tipo_turismo, :duracion_horas, :preferencias,
            :latitud, :longitud, CAST(:meteorologia AS jsonb), CAST(:lugares AS jsonb),
            :prompt_usado, :itinerario, :estado, :duracion_ms, :error_resumido,
            :modo_demo, :modo_modelo
        )
        RETURNING id;
        """
    )

    params = {
        "ciudad": registro.get("ciudad"),
        "pais": registro.get("pais"),
        "tipo_turismo": registro.get("tipo_turismo"),
        "duracion_horas": duracion,
        "preferencias": registro.get("preferencias"),
        "latitud": registro.get("latitud"),
        "longitud": registro.get("longitud"),
        "meteorologia": registro.get("meteorologia"),
        "lugares": registro.get("lugares"),
        "prompt_usado": registro.get("prompt_usado"),
        "itinerario": registro.get("itinerario"),
        "estado": registro.get("estado", "completado"),
        "duracion_ms": registro.get("duracion_ms"),
        "error_resumido": registro.get("error_resumido"),
        "modo_demo": bool(registro.get("modo_demo", False)),
        "modo_modelo": registro.get("modo_modelo", "cohere"),
    }

    result = db_session.execute(query, params)
    db_session.commit()
    return int(result.scalar_one())
```

2. Flask Route Handler with Sanitized Error Handling

Catch `ValueError` for client input errors and general exceptions for infrastructure failures without leaking stack traces or internal details:

```
# main.py
import logging
from flask import Flask, jsonify, request
from db import guardar_solicitud

app = Flask(__name__)
logger = logging.getLogger(__name__)


@app.route("/api/planificar", methods=["POST"])
def planificar():
    payload = request.get_json(silent=True) or {}

    try:
        solicitud_id = guardar_solicitud(db_session, payload)
        return (
            jsonify({"status": "success", "solicitud_id": solicitud_id}),
            201,
        )
```

```
    except ValueError as val_err:
        logger.warning("Validation failed: %s", val_err)
        return jsonify({"error": "Valor no válido para duracion_horas"}), 400

    except Exception:
        logger.exception("Database error occurred while saving solicitud")
        return (
            jsonify({"error": "No se pudo procesar la solicitud en el servidor"}),
            500,
        )
```

3. Isolated Pytest Integration Test

Use a UUID-tagged test record with cleanup executed inside a `finally` block to prevent database pollution across test runs:

```
# tests/test_db_integration.py
import uuid
import pytest
from sqlalchemy import text
from db import guardar_solicitud


def test_guardar_solicitud_lifecycle_cleanup(db_session):
    unique_tag = f"test_{uuid.uuid4().hex[:8]}"
    registro_test = {
        "ciudad": f"Valencia_{unique_tag}",
        "pais": "España",
        "tipo_turismo": "cultural",
        "duracion_horas": 4,  # Valid contract value: 2, 4, or 6
        "preferencias": "sin niños",
        "latitud": 39.4699,
        "longitud": -0.3763,
        "meteorologia": "{}",
        "lugares": "[]",
        "prompt_usado": "prompt demo",
        "itinerario": "itinerario demo",
        "estado": "completado",
        "duracion_ms": 120,
        "error_resumido": None,
        "modo_demo": True,
        "modo_modelo": "mock",
    }

    nuevo_id = None
    try:
        nuevo_id = guardar_solicitud(db_session, registro_test)
        assert isinstance(nuevo_id, int)
        assert nuevo_id > 0

        # Verify insertion and data integrity
        check_query = text(
            "SELECT ciudad, duracion_horas FROM solicitudes WHERE id = :id"
        )
        row = db_session.execute(check_query, {"id": nuevo_id}).mappings().one()
        assert row["ciudad"] == f"Valencia_{unique_tag}"
        assert row["duracion_horas"] == 4

    finally:
        if nuevo_id is not None:
            db_session.execute(
                text("DELETE FROM solicitudes WHERE id = :id"),
                {"id": nuevo_id},
            )
            db_session.commit()


def test_guardar_solicitud_invalid_duration_raises(db_session):
    registro_invalido = {
        "ciudad": "Valencia",
        "tipo_turismo": "cultural",
        "duracion_horas": 3,  # Invalid: must raise ValueError
    }

    with pytest.raises(ValueError, match="duracion_horas must be one of"):
        guardar_solicitud(db_session, registro_invalido)
```
